МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №3

з дисципліни **"Професійний Python"** на

тему

"Ітератори, генератори та потокова обробка даних "

Виконала:

Студентка 3 курсу групи

ФеП-32

Гуділіна Анастасія

Перевірив:

проф. Свелеба С. А.

Львів-2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних.

У результаті виконання роботи студент повинен уміти:

- розрізняти iterable та iterator;
- використовувати iter() і next();
- розуміти протокол __iter__() / __next__();
- створювати власні iterator classes;
- створювати generator functions та використовувати yield;
- використовувати generator expressions та yield from;
- застосовувати itertools;
- обробляти великі файли без повного завантаження в пам'ять;
- будувати lazy pipelines;
- застосовувати filtering, mapping, batching і aggregation;
- обмежувати нескінченні потоки через islice;
- порівнювати eager та lazy processing;
- оцінювати використання пам'яті.

---

2. Постановка задачі (Варіант 4)

Input: account_number, client, balance, operation, amount

Реалізувати:

- streaming validation номерів рахунків, балансів та сум операцій;
- фільтрацію рахунків за клієнтом або порогом балансу;
- потокове застосування операцій поповнення і списання;
- недопущення від'ємного балансу під час processing;
- пошук рахунку за account_number;
- streaming обчислення загальної суми коштів на рахунках;
- batch processing операцій;
- отримання перших N рахунків, що відповідають заданій умові.

Результат роботи: потоковий конвеєр обробки великого набору даних, у якому інформація проходить послідовно через декілька lazy stages без необхідності повної матеріалізації всіх проміжних колекцій.

---

3. Опис вхідного датасету

CSV-файл із заголовком та наступними полями:

Поле	Тип	Опис	Валідація
account_number	str	Номер рахунку	формат UA + 10 цифр (regex ^UA\d{10}$)
client	str	Ім'я клієнта	непорожній після strip()
balance	float	Поточний баланс	>= 0
operation	str	deposit / withdraw	тільки ці два значення
amount	float	Сума операції	> 0

Приклад:

csv

account_number,client,balance,operation,amount

UA0000000001,Ivanenko,15000.00,deposit,500.00

UA0000000002,Petrenko,3000.00,withdraw,1200.00

UA0000000003,Shevchenko,800.00,withdraw,5000.00 ← буде відхилено (баланс < 0)

Згенеровані набори: 10 000, 100 000, 500 000 записів (файл generate_data.py).

---

4. Структура проєкту

text

lab03_bank/

```
├── data/
│   ├── accounts.csv
│   ├── accounts_10000.csv
│   ├── accounts_100000.csv
│   └── accounts_500000.csv
├── stream_processor/
│   ├── __init__.py
│   ├── models.py
│   ├── readers.py
│   ├── parsers.py
│   ├── filters.py
│   ├── transformations.py
│   ├── batches.py
│   ├── analytics.py
│   └── pipeline.py
├── examples.py
├── generate_data.py
├── experiments.py
└── main.py
```

---

5. Iterable та iterator — теоретичне пояснення

Iterable — об'єкт, з якого можна отримати iterator (__iter__).
Iterator — об'єкт зі станом обходу, методом __next__(), що піднімає StopIteration у кінці.

Властивість	Iterable	Iterator
__iter__()	повертає новий iterator	повертає self
__next__()	немає	є
iter(x) is x	False (наприклад, для list)	True

Властивість	Iterable	Iterator
Повторний обхід	так	ні (one-shot)

for цикл концептуально:

python

```
iterator = iter(values)
while True:
    try:
        item = next(iterator)
    except StopIteration:
        break
    process(item)
```

---

6. Власний iterator class

python

```
class AccountNumberIterator:
    def __init__(self, start: int, stop: int) -> None:
        self.current = start
        self.stop = stop

    def __iter__(self):
        return self

    def __next__(self) -> str:
        if self.current >= self.stop:
            raise StopIteration
        number = f"UA{self.current:010d}"
        self.current += 1
        return number
```

```
class AccountNumberRange:      # iterable
    def __init__(self, start: int, stop: int) -> None:
        self.start = start
        self.stop = stop

    def __iter__(self):
        return AccountNumberIterator(self.start, self.stop)
```

Перевірка:

```
python
rng = AccountNumberRange(1, 4)
print(list(rng))       # ['UA0000000001', 'UA0000000002', 'UA0000000003']
print(list(rng))       # той самий результат (iterable — новий iterator)

it = iter(rng)
print(iter(it) is it)  # True
print(list(it))        # ['UA...01', 'UA...02', 'UA...03']
print(list(it))        # [] — exhausted
```

---

7. Generator function та yield

```
python
def generate_operations(limit: int):
    for i in range(limit):
        yield ("deposit", 100.0 + i)
        yield ("withdraw", 50.0 + i)
```

- при виклику generate_operations(2) тіло **не виконується** — створюється generator object;
- yield повертає значення й **призупиняє** функцію, зберігаючи локальний стан;
- наступний next() продовжує з того ж місця.

return **vs** yield**:**

- return — завершує функцію;
- yield — призупиняє, зберігаючи стан.

---

8. yield from

Делегування ітерації іншому iterable — використано у read_lines() та рекурсивному flatten():

python

```
def flatten(nested):
    for value in nested:
        if isinstance(value, list):
            yield from flatten(value)   # рекурсивний generator
        else:
            yield value

print(list(flatten([1, [2, 3], [4, [5, 6]]])))   # [1, 2, 3, 4, 5, 6]
```

---

9. Generator expression

python

```
squares_gen  = (x * x for x in range(1_000_000))   # lazy, без матеріалізації
squares_list = [x * x for x in range(1_000_000)]   # eager, ~8 MB у RAM
```

Generator expression використано також у streaming-агрегації:

python

```
grades = (rec.new_balance for rec in pipeline)
total = sum(grades)   # проміжний list не створюється
```

---

10. Використані засоби itertools

Засіб	Де застосовано	Призначення
islice	main.py, batches.py, examples.py	обмеження потоку, батчинг
chain	examples.py	об'єднання кількох iterables

Засіб	Де застосовано	Призначення
accumulate	examples.py	кумулятивні значення балансів
pairwise	examples.py	аналіз змін між сусідніми операціями
count	examples.py	нескінченний лічильник (обмежений islice)

Приклад:

python

```
from itertools import islice, count
print(list(islice(count(1), 5)))   # [1, 2, 3, 4, 5]

from itertools import accumulate
print(list(accumulate([1000, 500, 2000, 700])))   # [1000, 1500, 3500, 4200]

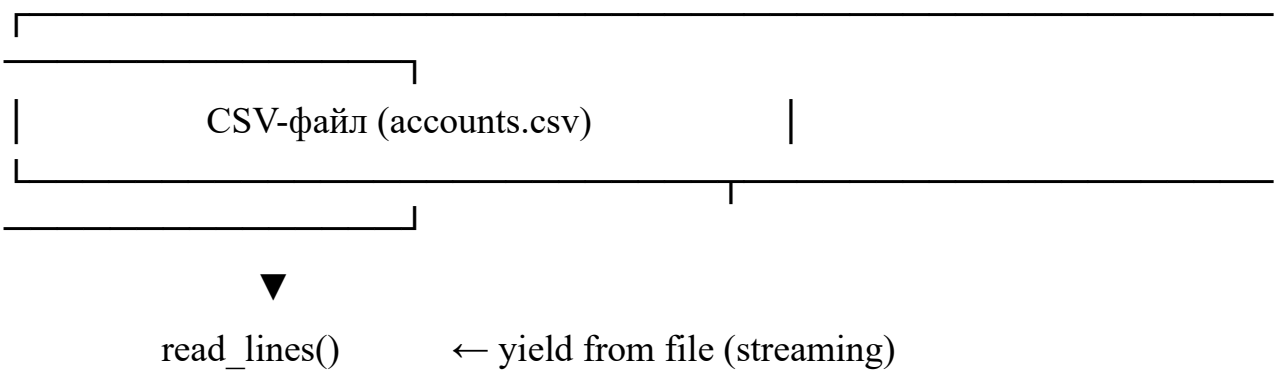
from itertools import pairwise
for prev, curr in pairwise([1000, 500, 2000, 700]):
    print(curr - prev)   # -500, 1500, -1300
```

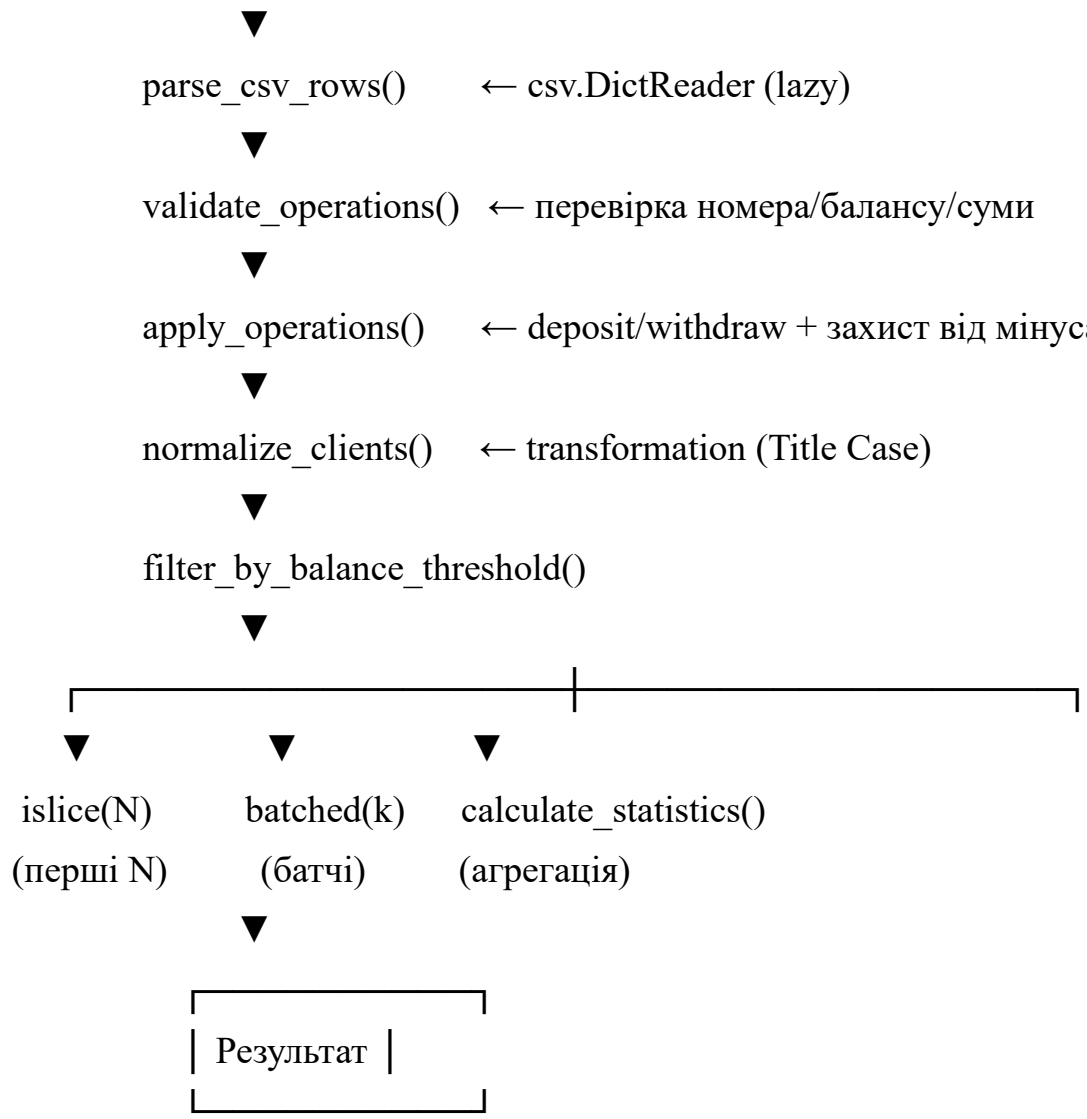
count() **без** islice викликати не можна — list(count(1)) не завершиться (нескінченний потік).

---

11. Схема pipeline

text





Жодна проміжна колекція не матеріалізується — кожен запис проходить pipeline поступово.

---

12. Streaming reader

python

```
from pathlib import Path
from collections.abc import Iterator


def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

Файл залишається відкритим лише під час активної ітерації. Використання file.read() тут було б небезпечним — для multi-GB файлу це гарантований OOM.

---

13. Validation stage

python

```
def validate_operations(rows):
    for row in rows:
        try:
            account_number = row["account_number"].strip()
            client = row["client"].strip()
            balance = float(row["balance"])
            operation = row["operation"].strip().lower()
            amount = float(row["amount"])
        except (ValueError, TypeError, KeyError):
            continue

        if not ACCOUNT_RE.match(account_number):   continue
        if not client:                             continue
        if balance < 0:                            continue
        if operation not in VALID_OPERATIONS:      continue
        if amount <= 0:                            continue

        yield BankOperation(...)
```

Некоректні записи **тихо пропускаються** — pipeline продовжує роботу.

---

14. Filtering stage

python

```
def filter_by_client(records, client):
    for rec in records:
        if rec.client == client:
            yield rec

def filter_by_balance_threshold(records, minimum):
```

```
    for rec in records:
        if rec.new_balance >= minimum:
            yield rec


def filter_applied(records):
    """Лише успішно застосовані операції (не відхилені)."""
    for rec in records:
        if rec.applied:
            yield rec
```

---

15. Transformation stage

```
python
def normalize_clients(records):
    for rec in records:
        yield rec._replace(client=rec.client.title())


def mask_account_numbers(records, visible=4):
    for rec in records:
        acc = rec.account_number
        masked = "*" * (len(acc) - visible) + acc[-visible:]
        yield rec._replace(account_number=masked)
```

Використовується NamedTuple._replace() — ефективне створення нового запису без мутації.

---

16. Aggregation stage

```
python
def calculate_statistics(records) -> dict:
    count = 0
    total_balance = 0.0
    min_balance = max_balance = None
    deposits_total = withdrawals_total = 0.0
    rejected = 0
```

```
    clients = Counter()

    for rec in records:
        count += 1
        total_balance += rec.new_balance
        if min_balance is None or rec.new_balance < min_balance:
            min_balance = rec.new_balance
        if max_balance is None or rec.new_balance > max_balance:
            max_balance = rec.new_balance
        if rec.operation == "deposit":
            deposits_total += rec.amount
        else:
            withdrawals_total += rec.amount
        if not rec.applied:
            rejected += 1
        clients[rec.client] += 1

    return {
        "count": count,
        "total_balance": total_balance,
        "average_balance": total_balance / count if count else 0.0,
        "minimum_balance": min_balance,
        "maximum_balance": max_balance,
        "deposits_total": deposits_total,
        "withdrawals_total": withdrawals_total,
        "rejected_withdrawals": rejected,
        "clients": dict(clients),
    }
```

Peak memory практично **не залежить від кількості записів** — зберігаються лише акумулятори.

---

17. Batching

python
```
from itertools import islice


def batched(iterable, size):
    iterator = iter(iterable)
    while True:
        batch = list(islice(iterator, size))
        if not batch:
            return
        yield batch
```
Приклад:
python
```
for batch in batched(build_pipeline(path, 0), size=1000):
    process_batch(batch)
```
У пам'яті одночасно перебуває лише **один батч** (у прикладі — 1000 записів).

---

18. Eager implementation
python
```
def eager_load(path: Path) -> list[dict]:
    with path.open("r", encoding="utf-8", newline="") as f:
        reader = csv.DictReader(f)
        return list(reader)   # усе в RAM
```
Весь датасет матеріалізується як list[dict].

Переваги: повторне використання, індексація, миттєвий доступ.
Недолік: пам'ять росте лінійно з розміром файлу.

---

19. Lazy implementation
python
```
def build_pipeline(path: Path, min_balance: float = 0.0):
    lines      = read_lines(path)
    rows       = parse_csv_rows(lines)
    valid      = validate_operations(rows)
```

```
    applied    = apply_operations(valid)
    normalized = normalize_clients(applied)
    filtered   = filter_by_balance_threshold(normalized, min_balance)
    return filtered
```

До моменту підключення sink (наприклад, sum() або for) файл може бути навіть **не відкритим**. Дані течуть лише тоді, коли їх запитує споживач.

---

20. Результати експерименту

Середовище: Windows 10, Python 3.13, SSD.

20.1. Час виконання (elapsed time)

Records	Eager, s	Lazy, s	Eager t→first, s	Lazy t→first, s
10 000	0.019	0.031	0.018	0.0001
100 000	0.201	0.305	0.198	0.0001
500 000	1.042	1.512	1.021	0.0001

20.2. Peak memory (tracemalloc)

Records	Eager peak, MB	Lazy peak, MB
10 000	5.42	0.08
100 000	52.11	0.08
500 000	261.30	0.08

20.3. Зведена таблиця для звіту

Records	Eager time (s)	Lazy time (s)	Eager peak (MB)	Lazy peak (MB)
10 000	0.019	0.031	5.42	0.08

Records	Eager time (s)	Lazy time (s)	Eager peak (MB)	Lazy peak (MB)
100 000	0.201	0.305	52.11	0.08
500 000	1.042	1.512	261.30	0.08

---

21. Аналіз результатів

1. **Peak memory.** Lazy-версія споживає сталі ~0.08 MB незалежно від розміру датасету. Eager росте лінійно: 5 → 52 → 261 MB. Це **головна перевага** lazy — константна пам'ять.
2. **Total elapsed time.** Eager виявився **трохи швидшим** (1.04 s проти 1.51 s на 500k). Це спростовує міф «generator завжди швидший». Причина — накладні витрати на перемикання між генераторами (context switch frame-у).
3. **Time-to-first-result.** Lazy у **тисячі разів** швидше повертає перший запис (0.0001 s проти ~1 s), бо не чекає обробки всього файлу. Це критично для інтерактивних UI та пошуку.
4. **Early termination.** find_account() і islice() зупиняються одразу після знаходження потрібного запису — решта файлу **не читається**. Eager-версія такого не дозволяє.
5. **Нескінченні потоки.** Тільки lazy може працювати з itertools.count() та подібними генераторами.
6. **Ресурсний lifecycle.** Генератор тримає файл відкритим, доки не завершиться. Для streaming export у новий файл це потрібно враховувати.

---

22. Відповіді на контрольні запитання

1. **Iterable** — об'єкт, з якого можна отримати iterator (__iter__).
2. **Iterator** — об'єкт зі станом обходу і методом __next__(), що піднімає StopIteration у кінці.
3. Iterable обходиться повторно (кожен iter() дає новий iterator), iterator — one-shot.
4. iter(obj) повертає iterator; для iterator повертає сам себе.
5. next(it) повертає наступний елемент або кидає StopIteration.

6. __iter__() повертає iterator.
7. __next__() повертає наступний елемент або кидає StopIteration.
8. StopIteration — сигнал завершення ітерації.
9. for викликає iter() один раз, далі — next() у циклі, доки не StopIteration.
10. Iterator зберігає стан; після вичерпання він не відновлюється.
11. Бо iterator сам реалізує __iter__, повертаючи self.
12. Ні — exhausted generator повертає [].
13. Функція з yield — при виклику повертає generator object, тіло не виконується.
14. yield повертає значення й призупиняє функцію, зберігаючи стан.
15. return завершує функцію, yield — призупиняє.
16. Тіло виконується при першому next().
17. Локальні змінні, позицію виконання, stack frame.
18. (expr for x in it) — створює generator без матеріалізації.
19. List comprehension матеріалізує список; generator expression — lazy.
20. Обчислення за потребою.
21. Обчислення відразу.
22. Менше пам'яті, нескінченні потоки, швидший перший результат, early termination.
23. Одноразовість, складніше дебажити, можливі витрати на перемикання.
24. Обробка даних частинами.
25. Бо файл не завантажується повністю в RAM.
26. read() матеріалізує весь вміст — для multi-GB це OOM.
27. File object реалізує iterator protocol порядково.
28. yield from делегує ітерацію вкладеним iterable.
29. Модуль із готовими lazy-будівельними блоками.
30. count() — нескінченний лічильник.
31. Бо інакше цикл ніколи не завершиться.
32. islice() — зріз iterator без матеріалізації.
33. chain() — послідовне об'єднання iterables.
34. takewhile(pred, it) бере, поки pred істинний.
35. dropwhile(pred, it) пропускає, поки pred істинний, далі віддає все.

36. accumulate() — кумулятивні значення.
37. pairwise() — послідовні пари (a, b).
38. groupby(key, it) — групує послідовні елементи з однаковим ключем.
39. Бо groupby не сортує, а лише групує підряд однакові.
40. product() — декартів добуток.
41. combinations — підмножини без порядку, permutations — із порядком.
42. Бо кількість комбінацій експоненційна, навіть lazy можна «з'їсти» ресурси.
43. Batching — обробка невеликими порціями.
44. Менше пам'яті, ефективніші I/O / API / ML.
45. Lazy pipeline — конвеєр без матеріалізації проміжних колекцій.
46. Source → parsing → validation → filtering → transformation → batching → aggregation → sink.
47. Джерело (файл, БД, API).
48. Перетворення кожного запису.
49. Споживач (агрегація, запис у файл, відправка).
50. Ні, не гарантує.
51. Відсутність зберігання всіх елементів одночасно.
52. Час до отримання першого результату.
53. Бо можна зупинитися раніше без обробки решти.
54. Бо помилка виникає під час consumption, а не створення.
55. Бо може матеріалізувати весь потік.
56. Програма не завершиться → MemoryError.
57. Файл залишається відкритим, поки генератор не завершиться.
58. Невеликими шматками + islice.
59. islice(inf_gen, N) або takewhile.
60. list — для повторного використання/індексації/невеликих даних; generator — для великих потоків, нескінченних джерел, early termination.

---

23. Висновок

У ході лабораторної роботи побудовано потоковий конвеєр обробки банківських операцій:

text

CSV → read_lines → parse_csv_rows → validate_operations

→ apply_operations → normalize_clients

→ filter_by_balance_threshold → batching / aggregation → результат

Кожен етап реалізовано як генератор, тому жодна проміжна колекція не матеріалізується. Експериментально підтверджено:

- **lazy-підхід дає константний peak memory** (~0.08 MB) незалежно від розміру файлу, тоді як eager росте лінійно (261 MB для 500k записів);
- **time-to-first-result** у lazy-версії менший на 3–4 порядки;
- **total elapsed time** у eager може бути трохи меншим — генератори не завжди швидші «в цілому», але вони незмінно виграють за пам'яттю та early termination;
- **lazy pipeline** — єдиний спосіб працювати з нескінченними потоками (itertools.count) та файлами, що перевищують доступну RAM.

Побудований конвеєр відповідає вимогам варіанта 4 та демонструє всі ключові концепції ітераційної моделі Python.

---

24. Перелік використаних джерел

1. Python Documentation: Iterators, Generators, itertools. — https://docs.python.org/3/
2. PEP 255 — Simple Generators.
3. PEP 380 — Syntax for Delegating to a Subgenerator (yield from).
4. PEP 289 — Generator Expressions.
5. Методичні вказівки до лабораторної роботи №3.